

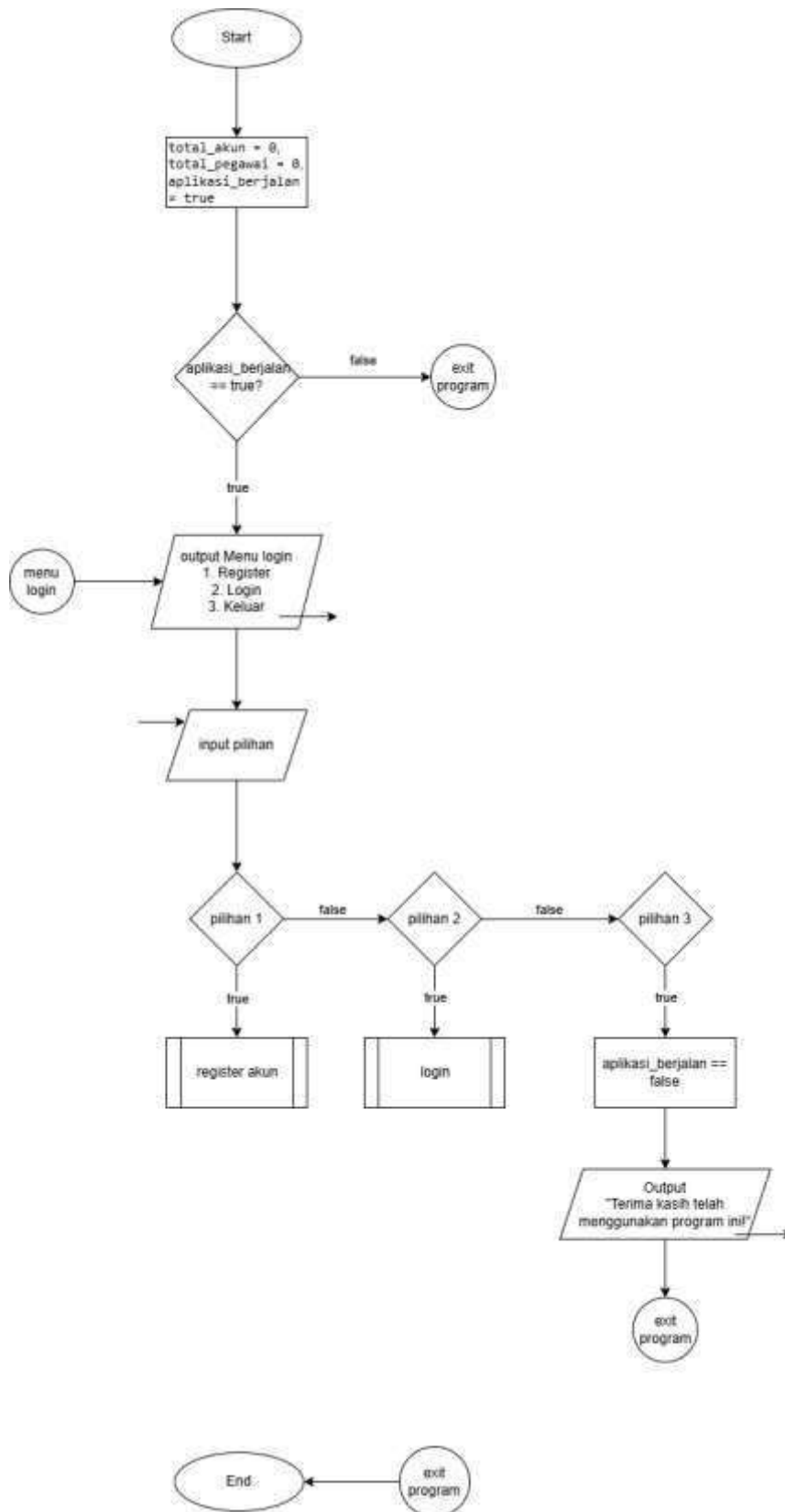
LAPORAN PRAKTIKUM
POSTTEST (4)
ALGORITMA PEMROGRAMAN LANJUT



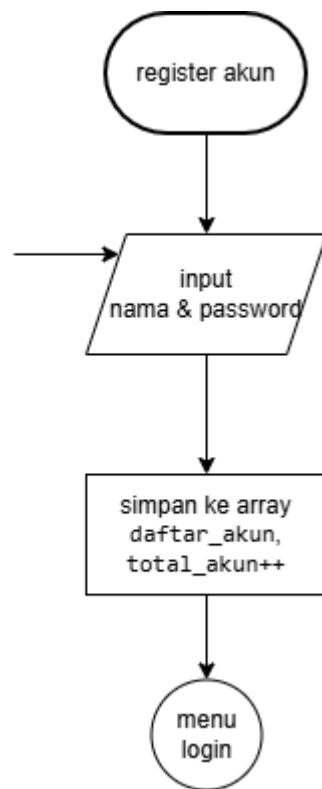
Disusun oleh:
Antung Hissyam (2509106092)
Kelas (C1 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

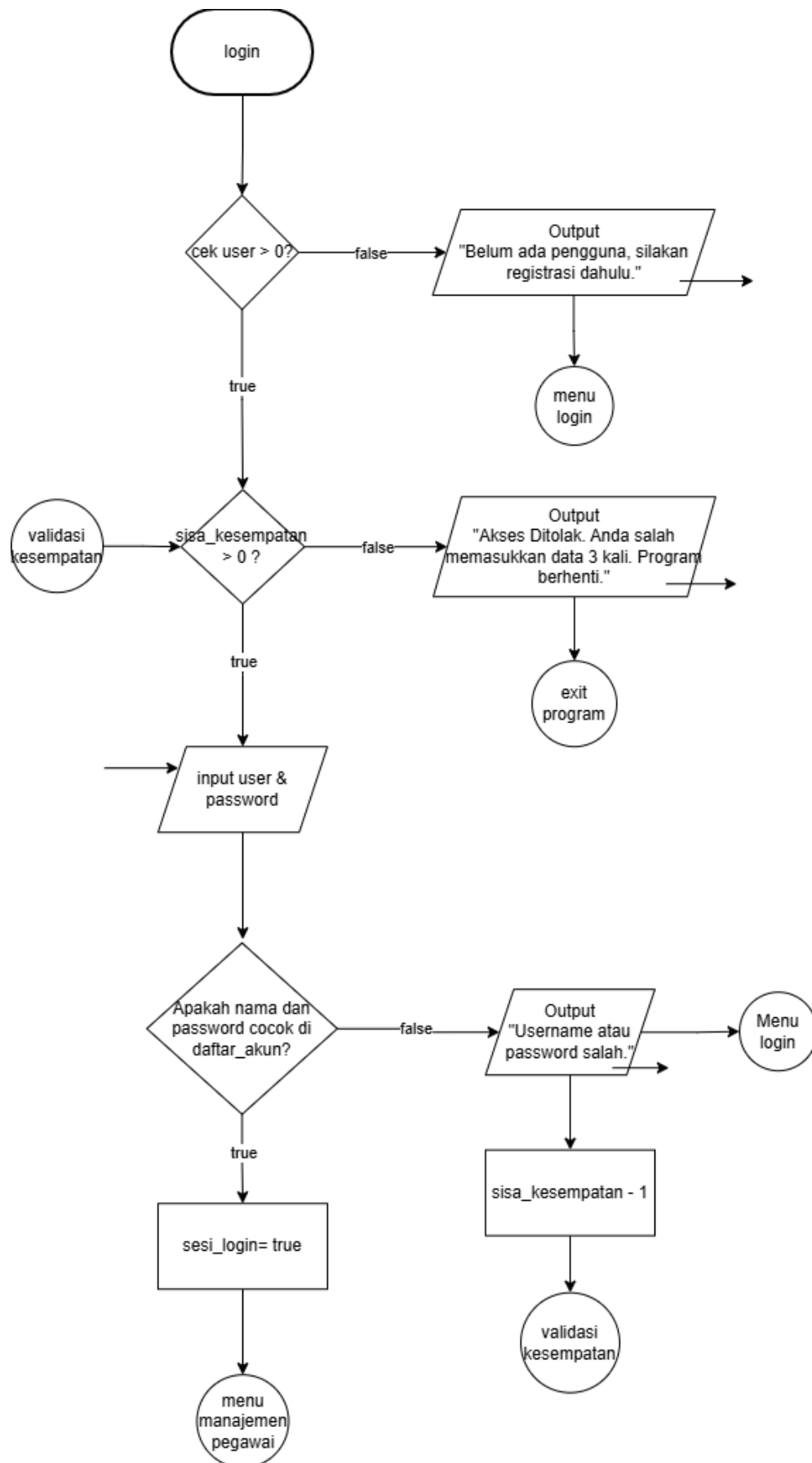
1. Flowchart



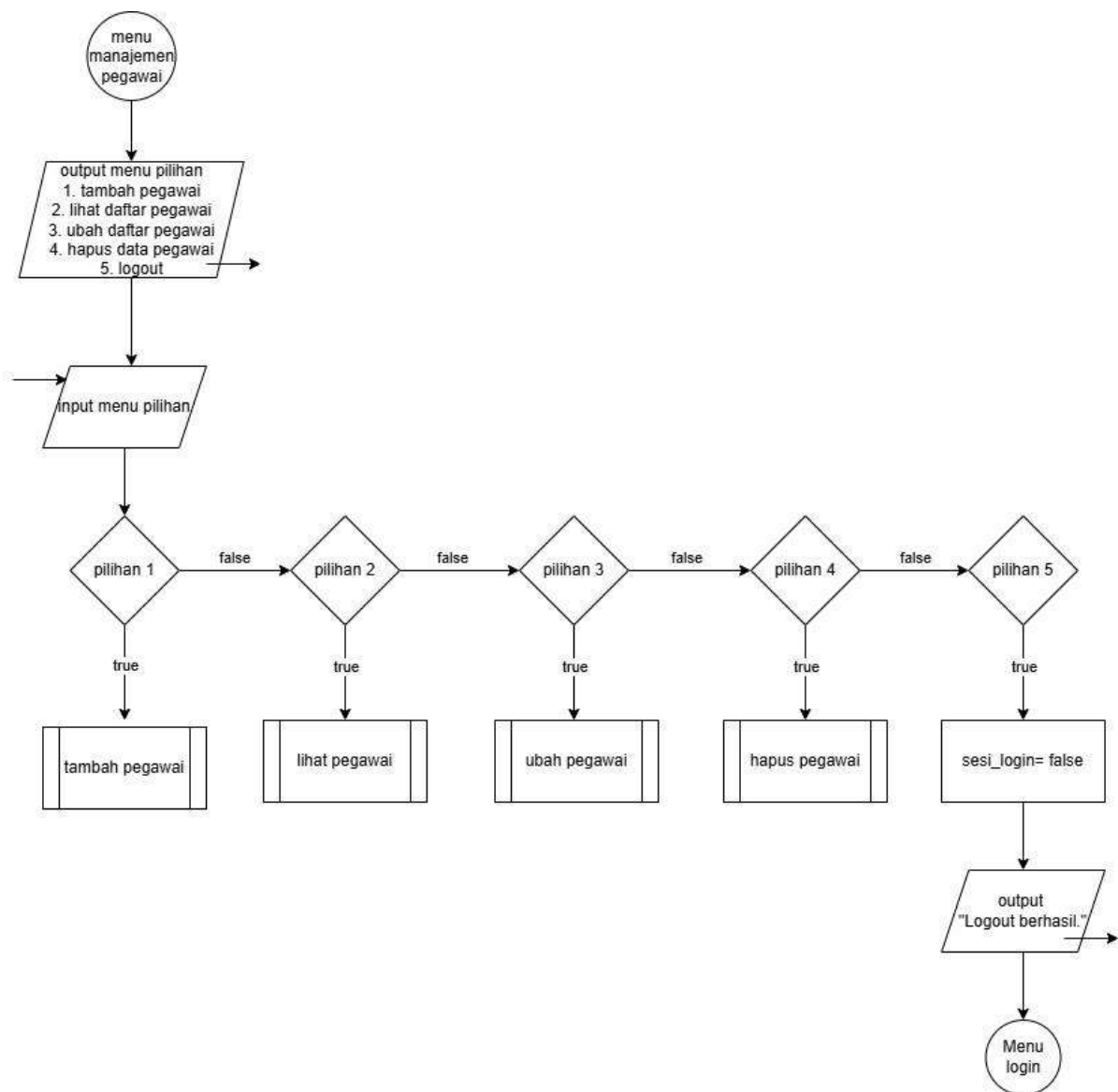
Gambar 1.1 Flowchart Menu Login.



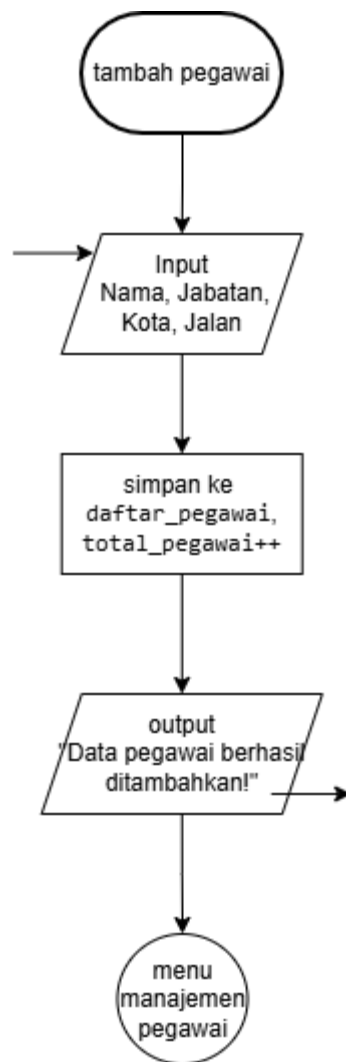
Gambar 1.2 Flowchart Fungsi Register Akun.



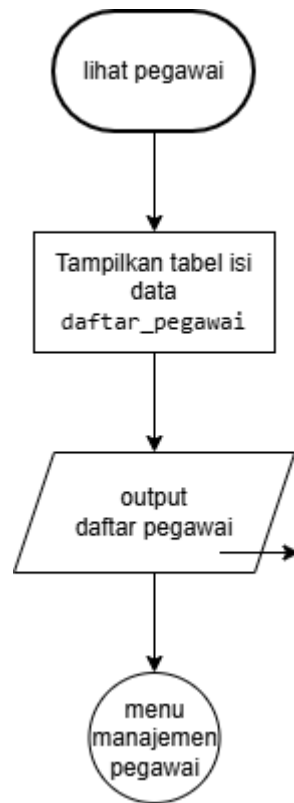
Gambar 1.3 Flowchart Fungsi Login.



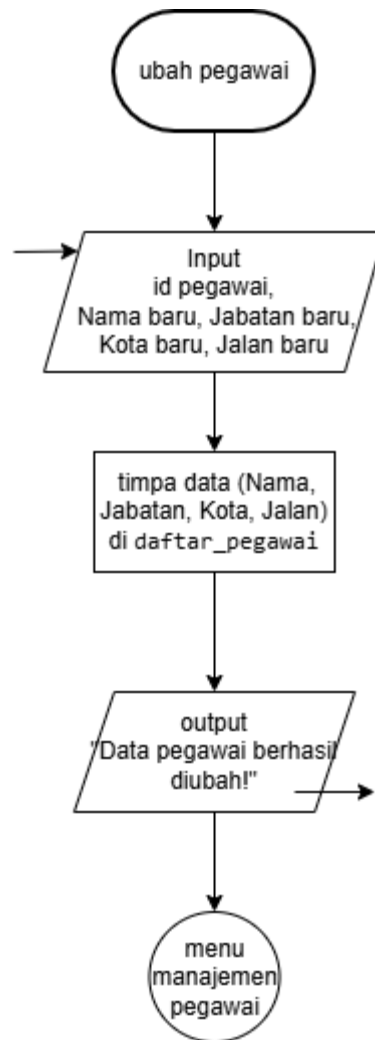
Gambar 1.4 Flowchart Menu CRUD.



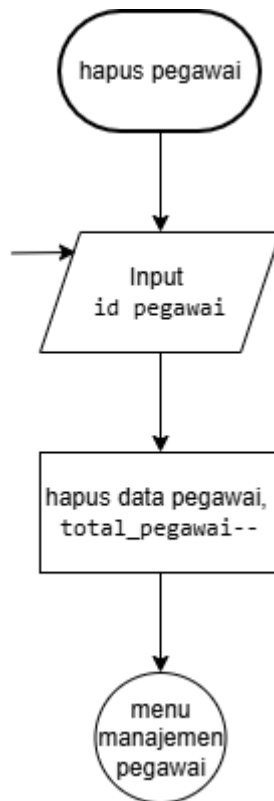
Gambar 1.5 Flowchart Fungsi Tambah Pegawai (Create).



Gambar 1.6 Flowchart Fungsi Lihat Pegawai (Read).



Gambar 1.7 Flowchart Fungsi Ubah Pegawai (Update).



Gambar 1.3 Flowchart Fungsi Hapus Pegawai (Delete).

Penjelasan Alur Flowchart (Algoritma Program Manajemen Pegawai)

Algoritma program berjalan dengan menampilkan menu utama. Jika memilih register, program memanggil fungsi dengan memberikan address dari total_akun. Saat proses login berhasil, alur masuk ke looping menu CRUD (Tambah, Lihat, Ubah, Hapus). Pada setiap fungsi CRUD, nilai total pegawai dimodifikasi secara langsung ke memorinya melalui perantara pointer, sehingga data selalu ter-update.

2. Deskripsi Singkat Program

Program aplikasi Manajemen Pegawai Kebab Cendana ini merupakan pengembangan langsung dari program praktikum modul sebelumnya. Fokus utama pada kode kali ini adalah implementasi konsep pointer pada program untuk efisiensi alokasi memori. Walaupun secara fitur program ini tetap beroperasi sebagai sistem manajemen data (CRUD) dengan sistem login, struktur di belakangnya telah diubah menggunakan prinsip pass-by-pointer. Perubahan ini ditandai dengan penggunaan operator address-of (&) ketika memanggil sebuah fungsi pendukung, serta penggunaan operator dereference (*) pada deklarasi parameter fungsinya. Hal ini memungkinkan fungsi untuk langsung memanipulasi variabel penghitung jumlah data dari luar scope-nya. Selain itu, program ini juga mendemonstrasikan penggunaan pointer di dalam struct, di mana sebuah penunjuk secara khusus dialokasikan untuk memegang alamat dari salah satu variabel data pegawai.

3. Source Code

A. Pointer pada Struct

Struct Pegawai dimodifikasi dengan penambahan string `*jabatan_ptr`. Saat fungsi `tambahPegawai` dipanggil, alamat memori dari input jabatan diberikan (address-of) kepada pointer tersebut, dan akan dicetak saat fungsi `read` dipanggil menggunakan operator dereference.

```
struct Pegawai {
    string nama_pegawai;
    string jabatan;
    Alamat lokasi;
    string *jabatan_ptr; // pointer yang menunjuk variabel
    'jabatan'
};

// ... di dalam fungsi tambahPegawai() ...
daftar_pegawai[indeks].jabatan_ptr =
&daftar_pegawai[indeks].jabatan;
```

B. Penerapan Address-of (&) dan Dereference (*) pada Parameter Fungsi

Pada setiap sub-program CRUD, program mengirimkan alamat memori (bukan copy nilai) ke fungsi tujuan. Fungsi tujuan menerima alamat tersebut sebagai pointer `*total_pegawai`, lalu melakukan dereference `(*total_pegawai)++` untuk menambah atau mengurangi nilai asli di `main()`.

```
// 1. Pada pemanggilan di int main() menggunakan address-of (&)
tambahPegawai(daftar_pegawai, &total_pegawai);

// 2. Pada deklarasi fungsi menggunakan dereference (*) parameter
void tambahPegawai(Pegawai daftar_pegawai[], int *total_pegawai) {
    // ...
    (*total_pegawai)++; // ngubah nilai asli di memori
}
```

C. Re-Linking Pointer saat Shift Array (Logika Memori Lanjut)

Saat fitur `hapusPegawai` dipanggil, nilai elemen array digeser ke atas. Program melakukan pembaruan alamat memori pointer `jabatan_ptr` agar tidak mengacu pada alamat yang digeser (mencegah dangling pointer).

```
for (int i = index_pegawai - 1; i < batas - 1; i++) {
    daftar_pegawai[i] = daftar_pegawai[i + 1];

    // Mencegah error memory karena pergeseran array
    daftar_pegawai[i].jabatan_ptr =
&daftar_pegawai[i].jabatan;
}
```

4. Hasil Output

A. Output Menu Register dan Login Berhasil

Tampilan ini menunjukkan simulasi saat pengguna pertama kali mendaftarkan akun, kemudian berhasil melakukan login menggunakan akun tersebut.

```
=====
                        SISTEM PEGAWAI KEBAB CENDANA
=====

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 1

--- REGISTER ---
Masukkan Nama (Tanpa Spasi) : antung
Masukkan Password           : 092
[+] Akun berhasil didaftarkan! Silakan Login.

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama      : antung
Masukkan Password  : 092

[+] Login Berhasil! Selamat datang, antung.

=====
                        MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: █
```

Gambar 4.1 Output Register dan Login Berhasil

B. Output Gagal Login 3 Kali (Akses Ditolak)

Tampilan ini membuktikan bahwa fitur pembatasan akses berjalan dengan baik. Jika pengguna salah menginput data sebanyak 3 kali, program otomatis dihentikan.

```

--- MENU AUTENTIKASI ---
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 2

--- LOGIN ---
Masukkan Nama      : aweawd
Masukkan Password  : wda
[!] Nama atau Password salah! Sisa percobaan: 2

Masukkan Nama      : wad
Masukkan Password  : dwa
[!] Nama atau Password salah! Sisa percobaan: 1

Masukkan Nama      : wad
Masukkan Password  : adw

[X] Anda gagal login 3 kali. Program berhenti otomatis.

```

Gambar 4.2 Output Login Gagal dan Program Berhenti

C. Output Tambah Pegawai (Create)

Tampilan ini menunjukkan ketika pengguna (admin) berhasil masuk ke menu CRUD dan melakukan penambahan data pegawai beserta alamatnya (Nested Struct).

```

=====
              MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 1

--- TAMBAH PEGAWAI ---
Nama Pegawai : antung
Jabatan      : ceo
Kota         : samarinda
Jalan        : cendana
[+] Data pegawai berhasil ditambahkan!

```

Gambar 4.3 Output Tambah Data Pegawai

D. Output Tampilkan Data (Read)

Tampilan ini membuktikan bahwa data yang diinputkan sebelumnya berhasil disimpan di dalam array of struct dan ditampilkan dengan format tabel yang rapi.

```

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 2

--- DAFTAR PEGAWAI KEBAB CENDANA ---
-----
No.      | Nama                | Jabatan                | Alamat (Kota, Jalan)
-----
1        | antung              | ceo                    | samarinda, cendana
-----

```

Gambar 4.4 Output Tabel Daftar Pegawai

E. Output Ubah dan Hapus Data (Update & Delete)

Tampilan ini menunjukkan proses pencarian data berdasarkan nomor indeks. Data pegawai akan diubah jika memilih opsi 3, atau dihapus jika memilih opsi 4.

```

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 3

--- UBAH DATA PEGAWAI ---
Masukkan nomor pegawai yang akan diubah: 1
Masukkan nama baru : antoeng
Masukkan jabatan baru: ceo muda aselole
Masukkan kota baru : balikpapan
Masukkan jalan baru : sudirman
[+] Data berhasil diubah.

=====
MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 4

--- HAPUS DATA PEGAWAI ---
Masukkan nomor pegawai yang akan dihapus: 1
[+] Pegawai berhasil dihapus.

```

Gambar 4.5 Output Ubah dan Hapus Data

F. Output Pilihan Tidak Valid dan Keluar

Tampilan ini menunjukkan ketika pengguna menginput angka di luar daftar menu (contoh: angka 6), program akan menampilkan peringatan dan mengulang menu.

```
=====
                        MENU MANAJEMEN PEGAWAI
=====
1. Tambah Pegawai (Create)
2. Lihat Daftar Pegawai (Read)
3. Ubah Data Pegawai (Update)
4. Hapus Pegawai (Delete)
5. Logout
Pilihan: 6
[!] Pilihan tidak valid!
```

Gambar 4.6 Output Validasi Menu

G. Output Exit Program

Tampilan ini menunjukkan ketika pengguna memilih angka 4, program akan berhenti.

```
=====
SISTEM PEGAWAI KEBAB CENDANA (ADMIN)
=====
1. Register Akun Baru
2. Login
3. Keluar Aplikasi
Pilihan: 3
Aplikasi akan tertutup dalam 3...
Aplikasi akan tertutup dalam 2...
Aplikasi akan tertutup dalam 1...
Terima kasih telah menggunakan aplikasi ini!
```

Gambar 4.7 Output Program Selesai

5. Langkah-langkah GIT

5.1 GIT Add

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git add .
```

Perintah `git add .` berfungsi untuk menambahkan semua file yang baru dibuat atau file yang mengalami modifikasi ke dalam *staging area*.

5.2 GIT Commit

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git commit -m "PT-1"
[main f5782f9] PT-1
 2 files changed, 105 insertions(+)
 create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.cpp
 create mode 100644 post-test/post-test-apl-1/2509106092-AntungHissyam-PT-1.exe
```

Perintah `git commit` berfungsi untuk merekam dan menyimpan secara permanen perubahan file yang sudah berada di *staging area* ke dalam riwayat repositori lokal.

5.3 GIT Push

```
PS E:\Coding\praktikum-apl\post-test\post-test-apl-1> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 448.22 KiB | 6.90 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Perintah git push berfungsi untuk mengunggah semua *commit* atau perubahan yang ada di repositori lokal (komputer) menuju ke repositori *remote* yang ada di Github.